

GENERATORS AND OBJECTS Solutions

COMPUTER SCIENCE MENTORS 61A

October 14 – October 17, 2025

1 Generators

1. Write a generator function that takes in an iterator `it` and yields the running total of the elements produced by it.

```
def accumulate(it):  
    '''  
    >>> def all_ints():  
    ...     i = 0  
    ...     while True:  
    ...         yield i  
    ...         i += 1  
    >>> a = accumulate(all_ints())  
    >>> [next(a) for x in range(6)]  
    [0, 1, 3, 6, 10, 15]  
    '''
```

```
def accumulate(it):  
    sum = 0  
    while True:  
        sum += next(it)  
        yield sum
```

2. Define `all_sums`, a generator that iterates through all the possible sums of elements from `lst`. (Repeat sums are permitted.)

```
def all_sums(lst):  
    '''  
    >>> list(all_sums([]))  
    [0]  
    >>> list(all_sums([1, 2]))  
    [3, 2, 1, 0]  
    >>> list(all_sums([1, 2, 3]))  
    [6, 5, 4, 3, 3, 2, 1, 0]  
    >>> list(all_sums([1, 2, 7]))  
    [10, 9, 8, 7, 3, 2, 1, 0]  
    '''  
  
    if len(lst) == 0:  
        yield 0  
    else:  
        for sum_rest in all_sums(lst[1:]):  
            yield sum_rest + lst[0]  
            yield sum_rest
```

3. Write a generator that, given *m* (the amount of money you have), *pc* (the cost of one pear), and *ac* (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):
    '''
    >>> print(list(fruitOptions(10, 2, 5)))
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear ',
    'apple pear pear ', 'apple apple ']
    '''
    if _____:
        yield ' '
    if m >= pc:
        for _____:
            _____
    if m >= ac:
        for _____:
            _____

def fruitOptions(m, pc, ac):
    if m < pc and m < ac:
        yield ' '
    if m >= pc:
        for p in fruitOptions(m-pc, pc, ac):
            yield 'pear ' + p;
    if m >= ac:
        for a in fruitOptions(m-ac, pc, ac):
            yield 'apple ' + a;
```

2 Objects and Attributes

1. Let's build a Bear using OOP!

Bear instances should have an attribute `name` that holds the name of the bear. The Bear class should have an attribute `bears`, a list that stores the name of each bear.

```
>>> oski = Bear('Oski')
>>> oski.name
'Oski'
>>> Bear.bears
['Oski']
>>> winnie = Bear('Winnie')
>>> Bear.bears
['Oski', 'Winnie']
```

```
class Bear:
```

```
    bears = []
    def __init__(self, name):
        self.name = name
        Bear.bears.append(self.name)
```

Note that `self.bears.append(self.name)` also works, but just doing `bears.append(self.name)` will result in an error!

There is no `bears` variable in the `__init__` function frame.

2. Implement the classes so the following code runs.

```
'''
>>> p = Plant()
>>> p.height
1
>>> p.materials
[]
>>> p.absorb()
>>> p.materials
[|Sugar|]
>>> Sugar.sugars_created
1
>>> p.leaf.sugars_used
0
>>> p.grow()
>>> p.materials
[]
>>> p.height
2
>>> p.leaf.sugars_used
1
'''
```

```

class Plant:
    def __init__(self):
        self.leaf = Leaf(self)
        self.materials = []
        self.height = 1

    def absorb(self):
        self.leaf.absorb()

    def grow(self):
        for sugar in self.materials:
            sugar.activate()
            self.height += 1

class Leaf:
    def __init__(self, plant): # plant is a Plant instance
        self.alive = True
        self.sugars_used = 0
        self.plant = plant

    def absorb(self):
        if self.alive:
            self.plant.materials.append(Sugar(self, self.plant))

    def __repr__(self):
        return '|Leaf|'

class Sugar:
    sugars_created = 0

    def __init__(self, leaf, plant):
        self.leaf = leaf
        self.plant = plant
        Sugar.sugars_created += 1

    def activate(self):
        self.leaf.sugars_used += 1
        self.plant.materials.remove(self)

    def __repr__(self):
        return '|Sugar|'

```

3. **Musician** What would Python display? Write the result of executing the code and the prompts below. If a function is returned, write 'Function'. If nothing is returned, write 'Nothing'. If an error occurs, write 'Error'.

```
class Musician:
    popularity = 0
    def __init__(self, instrument):
        self.instrument = instrument
    def perform(self):
        print('a stellar ' + self.instrument + ' performance')
        self.popularity = self.popularity + 2
    def __repr__(self):
        return self.instrument

class BandLeader(Musician):
    def __init__(self):
        self.band = []
    def recruit(self, musician):
        self.band.append(musician)
    def perform(self, song):
        for m in self.band:
            m.perform()
        Musician.popularity += 1
        print(song)
    def __str__(self):
        return 'Here's the band!'
    def __repr__(self):
        band = ''
        for m in self.band:
            band += str(m) + ' '
        return band[:-1]

miles = Musician('trumpet')
goodman = Musician('clarinet')
ellington = BandLeader()
```

Some Quick Refreshers

Defining attributes: Instance attributes are defined with the `self.attr_name` notation (usually in `__init__` but could be elsewhere like in this problem). Class attributes are defined outside of methods in the body of the class definition, like the variable `popularity` in the class `Musician`.

Accessing attributes: Instance attributes are referred to using `self.attr_name`. Class attributes can be referred to using `classname.attr_name` or `self.attr_name` (Note: using the latter will only work if there are no instance attributes bound with the name `attr_name`).

Before running any of the code below, `miles` and `goodman` are set to the musicians created as a result of calling the `__init__` constructor method in `Musician`. `ellington` uses `BandLeader`'s `__init__` method, since `BandLeader` is the subclass and has `__init__` defined.

```
>>> ellington.recruit(goodman)
>>> ellington.perform()
```

Error

`ellington.recruit(goodman)` adds `goodman` to the end of `ellington`'s instance attribute, `band`. Then, `ellington` checks its class (`BandLeader`) for the `perform()` method. But this `perform()` is expecting an argument, so this errors.

```
>>> ellington.perform('sing, sing, sing')
```

a rousing clarinet performance
sing, sing, sing

Using the same `perform()` method, now providing the correct number of arguments. First, going through the band list, `goodman` calls its `perform()` method, which is defined in `Musician`. Here, we print `'a rousing' + goodman's instrument + 'performance'`, and then `goodman's self.popularity = self.popularity + 2` happens. The `self.popularity` on the right of the equal sign is `Musician.popularity` because `goodman` doesn't have its own instance attribute named `popularity` yet; then it becomes `self.popularity = 0 + 2`, and this creates the instance attribute `popularity` for `goodman`. Then `Musician.popularity`, the class attribute, is incremented by 1.

```
>>> goodman.popularity, miles.popularity
```

```
(2, 1)
```

First, we try to get the value of `goodman.popularity`. In our environment diagram, we see that `goodman` has the instance variable `popularity` already defined. Therefore, we get that value, 2, back. Then, we try to access `miles.popularity`. In this case, `miles` doesn't have a `popularity` instance variable defined, so we default to the class variable. There, we see it defined as 1, so we get that value. Finally, since commas in Python define a tuple, we return the two values as `(2, 1)`.

```
>>> ellington.recruit(miles)
>>> ellington.perform('caravan')
```

```
a rousing clarinet performance
a rousing trumpet performance
caravan
```

First, we call `ellington.recruit(miles)`. This appends `miles` to `ellington`'s instance variable, `band`. After that, we call `ellington.perform('caravan')`. Similar to the previous call on `perform`, we will loop through all of the values in `ellington.band`, calling their `perform` methods in order. This causes the first two lines to be printed. Next, we increment `Musician.popularity` (the class variable of `Musician` called `popularity`). Lastly, we print the `song` variable that was passed in, completing the last line.

```
>>> ellington.popularity, goodman.popularity, miles.popularity
```

```
(2, 4, 3)
```

```
>>> print(ellington)
```

Here's the band!

`print()` expects the string representation of `ellington`, which is given by calling the `__str__()` method of `ellington`. `ellington` checks to see if `BandLeader` has a `__str__()` method, which it does. So, `print(ellington)` then becomes `print('Here's the band!')`.

```
>>> ellington
```

```
clarinet trumpet
```

When prompting for `ellington`'s value, we return the representation of `ellington` given by `__repr__()`. So, we call `BandLeader`'s `__repr__()` method.